

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

Отчет

по лабораторной работе № 9

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Поиск расстояний в графе»

Выполнили

студенты группы 24ВВВ2:

Макаров Я.С.

Бегичев Д.Д.

Родионов К.Е.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться искать расстояние между вершинами в графе.

Лабораторное задание

Задание 1

Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.

Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++

*Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

Задание 2*

*Реализуйте процедуру поиска расстояний на основе обхода в глубину

*Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности

*Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

Описание метода решения

Метод решения основан на алгоритмах обхода графов в ширину (BFS) и глубину (DFS) с использованием двух различных структур данных для представления графа: матрицы смежности и списков смежности.

Алгоритм BFS реализует классический поиск в ширину с использованием очереди, где обход начинается со стартовой вершины и последовательно охватывает все вершины на текущем уровне расстояния перед переходом к следующим уровням. Для матричного представления поиск смежных вершин требует полного перебора всех вершин в строке матрицы, что дает сложность $O(V)$ для каждой вершины, где V - количество вершин. В случае списков смежности доступ к соседним вершинам осуществляется напрямую, что обеспечивает сложность $O(k)$, где k - количество соседей вершины.

Алгоритм DFS использует стек для обхода в глубину, где происходит движение по одному пути до его полного исследования с последующим возвратом. В матричной реализации проверка соседей также требует полного перебора строки матрицы, тогда как списковая реализация позволяет эффективно итерироваться только по фактическим соседям вершины.

Во всех случаях BFS показывается более лучшее время поиска расстояния в сравнении с DFS.

Псевдокод

1-ое задание

алг обход_BFS_матрица_список

дано

число вершин `versh`

граф в виде матрицы смежности `matrica` размером `versh × versh`

граф в виде списков смежности `spiski`

стартовая вершина `start`

надо

сгенерировать случайный неориентированный граф

осуществить обход в ширину с построчным вычислением расстояний
для каждого представления

сравнить время обхода и вывести результаты

нач цел

`versh`

`matrica, spiski`

`dist_m, dist_s`

`start`

`время_старта, времени_конца`

// Построение списков смежности из матрицы

процедура `postroenieSpiskov()`

нач

очистить `spiski`

нц для `i` от 0 до `versh-1`

нц для `j` от 0 до `versh-1`

если `matrica[i][j] = 1` тогда добавить `j` в `spiski[i]`

```

    КЦ
КЦ
КОН

// Генерация графа
процедура generaciaGraf()
нач
инициализировать генератор случайных чисел
заполнить matrica случайным образом:
    нц для i от 0 до versh-1
        нц для j от i+1 до versh-1
            matrica[i][j], matrica[j][i] := случайное 0 или 1
        КЦ
    matrica[i][i] := 0
    КЦ
вызвать postroenieSpiskov()
КОН

// Обход BFS по матрице
функция bfsMatrica(start)
нач
создать dist_m длины versh с -1
создать очередь ochered
dist_m[start] := 0
положить start в ochered
записать время старта
пока ochered не пуста
    взять current из ochered
    нц для i от 0 до versh-1
        если matrica[current][i] = 1 и dist_m[i] = -1 тогда
            dist_m[i] := dist_m[current] + 1

```

положить i в `ochered`

КОН

КЦ

КЦ

записать время конца

вывести время работы в мкс

вернуть `dist_m`

КОН

// Обход BFS по спискам

функция `bfsSpiski(start)`

нач

создать `dist_s` длины `versh` с `-1`

создать очередь `ochered`

`dist_s[start] := 0`

положить `start` в `ochered`

записать время старта

пока `ochered` не пуста

 взять `current` из `ochered`

 для каждого соседа в `spiski[current]`

 если `dist_s[сосед] = -1` тогда

`dist_s[сосед] := dist_s[current] + 1`

 положить `сосед` в `ochered`

 КОН

КЦ

КЦ

записать время конца

вывести время работы в мкс

вернуть `dist_s`

КОН

```

// Вывод расстояний от вершины
процедура pechatDist(dist, start)
нач
вывести "Расстояния от вершины ", start
нц для i от 0 до versh-1
    вывести "Вершина ", i, ":", (dist[i] == -1 ? "-" : dist[i])
кц
кон

```

```

// Основная программа
нач
ввести versh
если versh ≤ 0 тогда вывод "Ошибка", завершить
создать граф с versh вершинами
вызвать generaciaGraf()
вызвать pechatMatrica()
вызвать pechatSpiski()
ввести start
если start не в диапазоне вывести "Ошибка"
иначе
    dist_m := bfsMatrica(start)
    pechatDist(dist_m, start)
    dist_s := bfsSpiski(start)
    pechatDist(dist_s, start)
кон
кон

```

2-ое задание

алг обход_BFS_DFS_матрица_списки

дано

число вершин `versh`

граф с матрицей смежности `matrica` и списками смежности `spiski`

стартовая вершина `start`

надо

выполнить обход в ширину (BFS) и в глубину (DFS) по двум
представлениям графа

замерить и вывести время работы каждого обхода

вывести расстояния от стартовой вершины

нач цел

`versh`

`matrica, spiski`

`dist_bfs_mat, dist_bfs_spis, dist_dfs_mat, dist_dfs_spis`

// Построение списков из матрицы

процедура `postroenieSpiskov()`

нач

очистить `spiski`

нц для `i` от 0 до `versh-1`

нц для `j` от 0 до `versh-1`

если `matrica[i][j] = 1` тогда добавить `j` в `spiski[i]`

кц

кц

кон

// Генерация случайного графа

процедура `generaciaGraf()`

нач

инициализировать генератор чисел

заполнить `matrica` случайно (без петель, симметрично)

вызвать postroenieSpiskov()

кон

// BFS по матрице

функция bfsMatrica(start)

нач

создать массив dist заполненный -1

создать очередь q

dist[start] := 0

положить start в q

начать замер времени

пока очередь не пуста

 извлечь вершину curr

 нц для i от 0 до versh-1

 если matrica[curr][i] = 1 и dist[i] = -1 тогда

 dist[i] := dist[curr] + 1

 положить i в q

 конец если

конец цикла

конец цикла

остановить замер времени и вывести

вернуть dist

кон

// BFS по спискам

функция bfsSpiski(start)

нач

создать массив dist заполненный -1

создать очередь q

dist[start] := 0

положить start в q

```
начать замер времени
пока очередь не пуста
    извлечь вершину curr
    для каждого соседа в spiski[curr]
        если dist[сосед] = -1 тогда
            dist[сосед] := dist[curr] + 1
            положить соседа в q
        конец если
    конец цикла
конец цикла
остановить замер времени и вывести
вернуть dist
конец
```

// DFS по матрице

функция dfsMatrica(start)

нач

создать массив dist заполненный -1

создать стек s

dist[start] := 0

положить start в s

начать замер времени

пока стек не пуст

извлечь вершину curr

нц для i от 0 до versh-1

если matrica[curr][i] = 1 и dist[i] = -1 тогда

dist[i] := dist[curr] + 1

положить i в s

конец если

конец цикла

конец цикла

остановить замер времени и вывести

вернуть dist

кон

// DFS по спискам

функция dfsSpiski(start)

нач

создать массив dist заполненный -1

создать стек s

dist[start] := 0

положить start в s

начать замер времени

пока стек не пуст

 извлечь вершину curr

 для каждого соседа в spiski[curr]

 если dist[сосед] = -1 тогда

 dist[сосед] := dist[curr] + 1

 положить соседа в s

 конец если

 конец цикла

конец цикла

остановить замер времени и вывести

вернуть dist

кон

// Вывод расстояний

процедура pechatDist(dist, start)

нач

вывести "Расстояния от вершины ", start

нц для i от 0 до versh-1

 вывести "Вершина ", i, ": ", dist[i] / "-" если -1

КЦ

КОН

// Основная программа

нач

ввести versh

если $\text{versh} \leq 0$ тогда вывести ошибку и задать по умолчанию 5

создать граф

вызвать generaciaGraf()

вызвать pechatMatrica()

вызвать pechatSpiski()

ввести start

если start в диапазоне

$\text{dist_bfs_mat} := \text{bfsMatrica}(\text{start})$

$\text{pechatDist}(\text{dist_bfs_mat}, \text{start})$

$\text{dist_bfs_spis} := \text{bfsSpiski}(\text{start})$

$\text{pechatDist}(\text{dist_bfs_spis}, \text{start})$

$\text{dist_dfs_mat} := \text{dfsMatrica}(\text{start})$

$\text{pechatDist}(\text{dist_dfs_mat}, \text{start})$

$\text{dist_dfs_spis} := \text{dfsSpiski}(\text{start})$

$\text{pechatDist}(\text{dist_dfs_spis}, \text{start})$

иначе

 вывести ошибку

КОН

КОН

Листинг

1-ое задание

```
#include <iostream>
#include <vector>
#include <queue>
#include <random>
#include <iomanip>
#include <chrono>

using namespace std;
using namespace std::chrono;

class Graf {
private:
    vector<vector<int>> matrica;
    vector<vector<int>> spiski;
    int versh;

    void postroenieSpiskov() {
        spiski.clear();
        spiski.resize(versh);
        for (int i = 0; i < versh; i++) {
            for (int j = 0; j < versh; j++) {
                if (matrica[i][j] == 1) {
                    spiski[i].push_back(j);
                }
            }
        }
    }

public:
    Graf(int n) : versh(n) {
        generaciaGraf();
    }
};
```

```

}

void generaciaGraf() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(0, 1);

    matrica.resize(versh, vector<int>(versh, 0));

    for (int i = 0; i < versh; i++) {
        for (int j = i + 1; j < versh; j++) {
            int rebro = dis(gen);
            matrica[i][j] = rebro;
            matrica[j][i] = rebro;
        }
    }
    postroenieSpiskov();
}

void pechatMatrica() {
    cout << "\nМатрица смежности:\n ";
    for (int i = 0; i < versh; i++)
        cout << setw(3) << i;
    cout << "\n";
    for (int i = 0; i < versh; i++) {
        cout << setw(3) << i;
        for (int j = 0; j < versh; j++)
            cout << setw(3) << matrica[i][j];
        cout << "\n";
    }
}

```

```

void pechatSpiski() {
    cout << "\nСписки смежности:\n";
    for (int i = 0; i < versh; i++) {
        cout << i << ": ";
        for (int sosed : spiski[i]) {
            cout << sosed << " ";
        }
        cout << "\n";
    }
}

```

```

vector<int> bfsMatrica(int start) {
    auto start_time = high_resolution_clock::now();

    vector<int> dist(versh, -1);
    queue<int> ochered;
    dist[start] = 0;
    ochered.push(start);

    while (!ochered.empty()) {
        int curr = ochered.front();
        ochered.pop();
        for (int i = 0; i < versh; i++) {
            if (matrica[curr][i] == 1 && dist[i] == -1) {
                dist[i] = dist[curr] + 1;
                ochered.push(i);
            }
        }
    }

    auto end_time = high_resolution_clock::now();
}

```

```
        auto duration = duration_cast<microseconds>(end_time -
start_time);

        cout << "Время обхода в ширь (матрица): " <<
duration.count() << " мкс\n";
```

```
        return dist;
    }
}
```

```
vector<int> bfsSpiski(int start) {
    auto start_time = high_resolution_clock::now();
```

```
    vector<int> dist(versh, -1);
    queue<int> ochered;
    dist[start] = 0;
    ochered.push(start);
```

```
    while (!ochered.empty()) {
        int curr = ochered.front();
        ochered.pop();
        for (int sosed : spiski[curr]) {
            if (dist[sosed] == -1) {
                dist[sosed] = dist[curr] + 1;
                ochered.push(sosed);
            }
        }
    }
}
```

```
    auto end_time = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end_time -
start_time);

    cout << "Время обхода в ширь (списки): " <<
duration.count() << " мкс\n";
```



```

        return dist;
    }

void pechatDist(const vector<int>& dist, int start) {
    cout << "\nРасстояния от вершины " << start << ":\n";
    for (int i = 0; i < versh; i++) {
        cout << "Вершина " << i << ": ";
        if (dist[i] == -1)
            cout << "-";
        else
            cout << dist[i];
        cout << "\n";
    }
}

};

int main() {
    setlocale(LC_ALL, "Russian");
    int versh;

    cout << "Введите количество вершин графа: ";
    cin >> versh;
    if (versh <= 0) {
        cout << "Неверное количество вершин! Используется значение по умолчанию: 5\n";
        versh = 5;
    }

    Graf graf(versh);

    graf.pechatMatrica();
    graf.pechatSpiski();
}

```

```

    int start;
    cout << "Введите начальную вершину для обхода в ширину: ";
    cin >> start;
    if (start >= 0 && start < versh) {
        vector<int> dist_m = graf.bfsMatrica(start);
        graf.pechatDist(dist_m, start);

        vector<int> dist_s = graf.bfsSpiski(start);
        graf.pechatDist(dist_s, start);
    }
    else {
        cout << "Неверная вершина!\n";
    }

    return 0;
}

```

2-ое задание

```

#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <random>
#include <chrono>
#include <iomanip>
using namespace std;
using namespace std::chrono;

class Graf {
private:
    vector<vector<int>> matrica;
    vector<vector<int>> spiski;

```

```

int versh;

void postroenieSpiskov() {
    spiski.clear();
    spiski.resize(versh);
    for (int i = 0; i < versh; i++) {
        for (int j = 0; j < versh; j++) {
            if (matrica[i][j] == 1) {
                spiski[i].push_back(j);
            }
        }
    }
}

public:

Graf(int n) : versh(n) {
    generaciaGraf();
}

void generaciaGraf() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(0, 1);
    matrica.resize(versh, vector<int>(versh, 0));
    for (int i = 0; i < versh; i++) {
        for (int j = i + 1; j < versh; j++) {
            int rebro = dis(gen);
            matrica[i][j] = rebro;
            matrica[j][i] = rebro;
        }
    }
    postroenieSpiskov();
}

```

```
}
```

```
void pechatMatrica() {  
    cout << "\nМатрица смежности:\n ";  
    for (int i = 0; i < versh; i++)  
        cout << setw(3) << i;  
    cout << "\n";  
    for (int i = 0; i < versh; i++) {  
        cout << setw(3) << i;  
        for (int j = 0; j < versh; j++)  
            cout << setw(3) << matrica[i][j];  
        cout << "\n";  
    }  
}
```

```
void pechatSpiski() {  
    cout << "\nСписки смежности:\n";  
    for (int i = 0; i < versh; i++) {  
        cout << i << ": ";  
        for (int sosed : spiski[i]) {  
            cout << sosed << " ";  
        }  
        cout << "\n";  
    }  
}
```

```
vector<int> bfsMatrica(int start) {  
    auto time_start = high_resolution_clock::now();  
  
    vector<int> dist(versh, -1);  
    queue<int> q;  
    dist[start] = 0;
```

```

q.push(start);
while (!q.empty()) {
    int curr = q.front();
    q.pop();
    for (int i = 0; i < versh; i++) {
        if (matrica[curr][i] == 1 && dist[i] == -1) {
            dist[i] = dist[curr] + 1;
            q.push(i);
        }
    }
}

auto time_end = high_resolution_clock::now();
cout << "Время ширины (матрица): " <<
duration_cast<microseconds>(time_end - time_start).count() << "
мкс\n";

return dist;
}

vector<int> bfsSpiski(int start) {
    auto time_start = high_resolution_clock::now();

    vector<int> dist(versh, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int curr = q.front();
        q.pop();
        for (int sosed : spiski[curr]) {
            if (dist[sosed] == -1) {
                dist[sosed] = dist[curr] + 1;
            }
        }
    }
}

```

```

        q.push(sosed);
    }
}

    }

    auto time_end = high_resolution_clock::now();
    cout << "Время ширины (списки): " <<
duration_cast<microseconds>(time_end - time_start).count() << "
мкс\n";

    return dist;
}

vector<int> dfsMatrica(int start) {
    auto time_start = high_resolution_clock::now();

    vector<int> dist(versh, -1);
    stack<int> s;
    dist[start] = 0;
    s.push(start);
    while (!s.empty()) {
        int curr = s.top();
        s.pop();
        for (int i = 0; i < versh; i++) {
            if (matrica[curr][i] == 1 && dist[i] == -1) {
                dist[i] = dist[curr] + 1;
                s.push(i);
            }
        }
    }
}

    auto time_end = high_resolution_clock::now();

```

```
        cout << "Время глубины (матрица): " <<
duration_cast<microseconds>(time_end - time_start).count() << "
мкс\n";
```

```
        return dist;
```

```
    }
```

```
vector<int> dfsSpiski(int start) {
    auto time_start = high_resolution_clock::now();
```

```
    vector<int> dist(versh, -1);
```

```
    stack<int> s;
```

```
    dist[start] = 0;
```

```
    s.push(start);
```

```
    while (!s.empty()) {
```

```
        int curr = s.top();
```

```
        s.pop();
```

```
        for (int sosed : spiski[curr]) {
```

```
            if (dist[sosed] == -1) {
```

```
                dist[sosed] = dist[curr] + 1;
```

```
                s.push(sosed);
```

```
            }
```

```
        }
```

```
    }
```

```
    auto time_end = high_resolution_clock::now();
```

```
    cout << "Время глубины (списки): " <<
duration_cast<microseconds>(time_end - time_start).count() << "
мкс\n";
```

```
    return dist;
```

```
}
```

```

void pechatDist(const vector<int>& dist, int start) {
    cout << "\nРасстояния от вершины " << start << ":\n";
    for (int i = 0; i < versh; i++) {
        cout << "Вершина " << i << ": ";
        if (dist[i] == -1)
            cout << "-";
        else
            cout << dist[i];
        cout << "\n";
    }
}
};

```

```

int main() {
    setlocale(LC_ALL, "Russian");
    int versh;
    cout << "Введите количество вершин графа: ";
    cin >> versh;

    if (versh <= 0) {
        cout << "Неверное количество вершин! Установлено
значение по умолчанию: 5\n";
        versh = 5;
    }

    Graf graf(versh);

    graf.pechatMatrica();
    graf.pechatSpiski();

    int start;
    cout << "Введите начальную вершину: ";

```



```
cin >> start;

if (start >= 0 && start < versh) {
    auto dist_bfs_mat = graf.bfsMatrica(start);
    graf.pechatDist(dist_bfs_mat, start);

    auto dist_bfs_spis = graf.bfsSpiski(start);
    graf.pechatDist(dist_bfs_spis, start);

    auto dist_dfs_mat = graf.dfsMatrica(start);
    graf.pechatDist(dist_dfs_mat, start);

    auto dist_dfs_spis = graf.dfsSpiski(start);
    graf.pechatDist(dist_dfs_spis, start);
}
else {
    cout << "Неверная вершина!\n";
}

return 0;
}
```

Результат работы программы

1-ое:

```
Консоль отладки Microsoft Vi  X + v
Введите количество вершин графа: 5

Матрица смежности:
  0 1 2 3 4
0 0 1 1 1 1
1 1 0 0 0 1
2 1 0 0 1 0
3 1 0 1 0 1
4 1 1 0 1 0

Списки смежности:
0: 1 2 3 4
1: 0 4
2: 0 3
3: 0 2 4
4: 0 1 3

Введите начальную вершину для обхода в ширину: 0
Время обхода в ширь (матрица): 18 мкс

Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 1
Вершина 2: 1
Вершина 3: 1
Вершина 4: 1
Время обхода в ширь (списки): 7 мкс

Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 1
Вершина 2: 1
Вершина 3: 1
Вершина 4: 1
```

2-ое:

Введите количество вершин графа: 5

Матрица смежности:

	0	1	2	3	4
0	0	0	1	0	0
1	0	0	0	0	1
2	1	0	0	0	0
3	0	0	0	0	0
4	0	1	0	0	0

Списки смежности:

0: 2

1: 4

2: 0

3:

4: 1

Введите начальную вершину: 0

Время ширины (матрица): 13 мкс

Расстояния от вершины 0:

Вершина 0: 0

Вершина 1: -

Вершина 2: 1

Вершина 3: -

Вершина 4: -

Время ширины (списки): 7 мкс

Расстояния от вершины 0:

Вершина 0: 0

Вершина 1: -

Вершина 2: 1

Вершина 3: -

Вершина 4: -

Время глубины (матрица): 3 мкс

Расстояния от вершины 0:

Вершина 0: 0

Вершина 1: -

Вершина 2: 1

Вершина 3: -

Вершина 4: -

Время глубины (списки): 2 мкс

Расстояния от вершины 0:

Вершина 0: 0

Вершина 1: -

Вершина 2: 1

Вершина 3: -

Вершина 4: -

Вывод

В ходе лабораторной работы было установлено, что алгоритм BFS демонстрирует более высокую эффективность по сравнению с DFS при решении задачи поиска расстояний в графе. BFS обеспечивает систематический обход вершин по уровням, что гарантирует нахождение кратчайшего пути до каждой достижимой вершины. В отличие от него, DFS может исследовать более длинные пути прежде, чем вернуться к ближайшим соседям, что делает его менее оптимальным для данной задачи. При использовании одинаковых структур данных BFS показал сопоставимое или лучшее время выполнения, подтвердив свою целесообразность для задач поиска расстояний. Наилучшие результаты показала комбинация BFS со

списками смежности, которая сочетает эффективность алгоритма с оптимальной структурой хранения графа.